

140. Le même jeu, deux fois

Une balle à la suite, trois balles par méthodes — le corps est le même.

1. Ce qu'il faut comprendre

À la suite, une balle demande quatre variables et huit lignes dans `main` : `if (versDroite) { x += 2; if (x > 150) versDroite = 0; }`, et ainsi de suite. Pour **une** balle, c'est parfait — 1427 octets.

Ci-dessous, trois balles **par méthodes**. Le corps de `avancer()` est **mot pour mot** celui de la version à la suite ; ce qui a changé, c'est qu'il est écrit une fois pour trois, et rangé avec les données sur lesquelles il travaille. 1956 octets, et quinze octets de mémoire.

La seule règle honnête : écrire à la suite tant que c'est un objet, nommer par une fonction quand le geste se répète, ranger dans une méthode quand les gestes et les données vont manifestement ensemble. Et ne pas choisir d'avance — recompiler, et lire le compte rendu.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **796 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	47 o	← la console, à l'allumage — la mise en route de la console
Balle::avancer()	267 o	← main() — ta fonction
main()	347 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
Tuiles	16 o	← CopierTuiles — 0 lettre sur 43, et 1 tuile dessinée
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

sprite() × 1	96 o	sur place, dans main()
image() × 1	6 o	sur place, dans main() + AttendreImage (28 o)

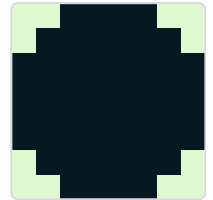
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : trois balles rebondissent sur les bords de l'écran,
// chacune à sa vitesse.
//
// Chaque balle est une struct : sa place (x, y), sa direction (versDroite,
// versBas : 1 ou 0) et sa vitesse (pas). La méthode avancer() est écrite
// UNE fois et sert aux trois balles.

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <sprite>   // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — BALLE

```
Tuile BALLE = {
    "..####..", ".#####.", "#####", "#####",
    "#####", "#####", ".#####.", "..####..",
};
```



à l'écran

3. UN ENREGISTREMENT — BALLE

```
struct Balle {
    uint8_t x, y, versDroite, versBas, pas;

    void avancer() {
        // En largeur : on avance dans sa direction ; au bord, on repart dans l'autre.
        if (versDroite) { x += pas; if (x > 150) versDroite = 0; } // bord droit → vers la gauche
        else { x -= pas; if (x < 10) versDroite = 1; } // bord gauche → vers la droite

        // En hauteur : pareil.
        if (versBas) { y += pas; if (y > 130) versBas = 0; } // en bas → remonte
        else { y -= pas; if (y < 10) versBas = 1; } // en haut → redescend
    }
};
```

4. UN TABLEAU — BALLES

```
Balle balles[3]; // 3 balles.
```

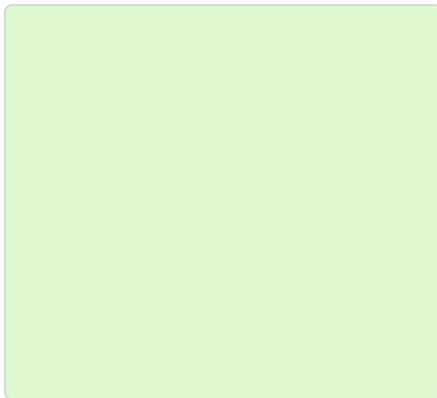
5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() { // Le jeu commence ici.
    for (uint8_t i = 0; i < 3; i++) { // Chaque balle i (0, 1, 2) :
        balles[i].x = 20 + i * 40; // x = 20, 60, 100
        balles[i].y = 30 + i * 20; // y = 30, 50, 70
        balles[i].versDroite = i % 2; // 0, 1, 0 : une sur deux part à droite
        balles[i].versBas = 1; // toutes descendent
        balles[i].pas = 1 + i; // vitesse 1, 2, 3 pixels par image
    }

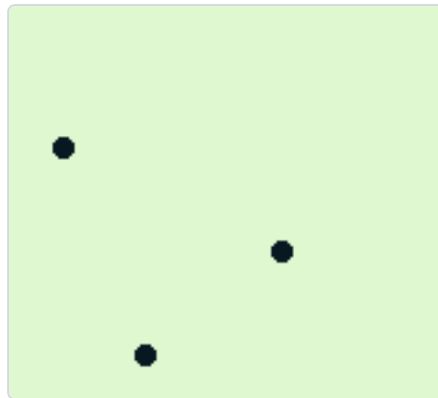
    while (true) { // Répète pour toujours :
        for (uint8_t i = 0; i < 3; i++) { // Chaque balle :
            balles[i].avancer(); // elle avance,
            sprite(i, balles[i].x, balles[i].y, BALLE); // on la pose (lutin numéro i).
        }
        image(); // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

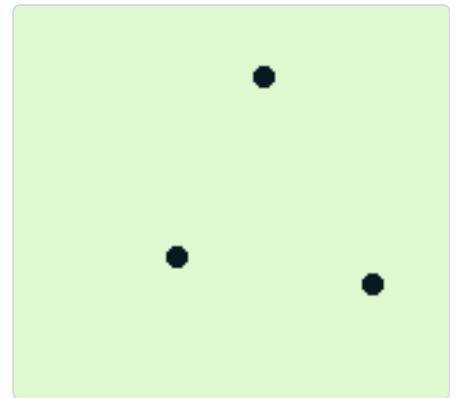
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



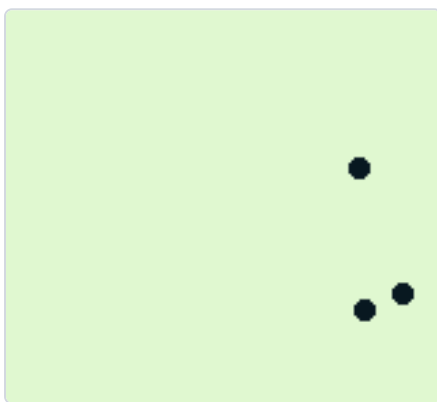
la toute première image — la console vient de démarrer — image 1



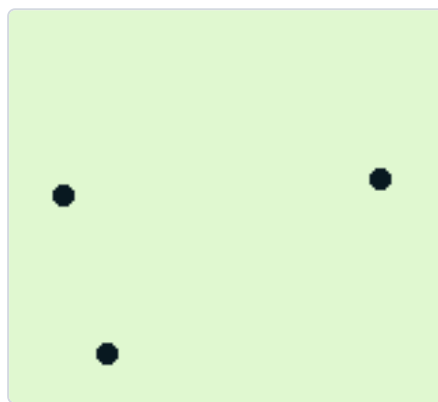
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Trois balles qui rebondissent, à trois vitesses différentes.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les trois balles bougent
- ✓ elles ne vont pas à la même vitesse
- ✓ aucune ne sort du cadre en trois cents images (0 écarts)
- ✓ la méthode sert les trois

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

urs et fonctions, une seule suite pour écrire une cartouche Game Boy en C++

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

BALLE

+ tuile 8 × 8

+ perso 16 × 16

+ perso 32 × 32

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

BALLE

tuile n° 44 — poser(x, y, BALLE)

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

×2

÷2

Agrandir...

Vider le dessin

Taille voulue :

✓ Créer la copie

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

BALLE ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

BALLE peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

Ensuite : 141. Ce que le compilateur refuse, et pourquoi — chapitre 24 : Aller au bout. · gameboy3 — écrire une cartouche Game Boy en C++